

5_Modelado

February 24, 2026

```
[1]: # Cargar los datos de entrenamiento
import numpy as np
train = np.load('../final_features/train.npy')

X_train = train[:, :-1]
y_train = train[:, -1]

print(np.shape(X_train))
print(np.shape(y_train))
```

(152, 10)

(152,)

```
[19]: # Definir los modelos de clasificación
from sklearn.linear_model import LogisticRegression
from sklearn.neural_network import MLPClassifier

algoritmos = {'LOGR': LogisticRegression(penalty='l2', solver='saga',
    ↪max_iter=1000, random_state=42),
              'MLP': MLPClassifier(hidden_layer_sizes=[8,4], activation='relu',
    ↪solver='sgd', batch_size='auto',
                                learning_rate='adaptive', learning_rate_init=0.
    ↪01, max_iter=1000, random_state=42)}

# Cross-validation interno en k=5 bolsas
from sklearn.model_selection import cross_val_score, KFold

results={}
for nombre, alg in algoritmos.items():
    results[nombre] = cross_val_score(alg, X_train, y_train,
    ↪cv=KFold(n_splits=5, shuffle=True, random_state=42))
    print(nombre + ': Accuracy: %0.4f +/- %0.4f' % (results[nombre].mean(),
    ↪results[nombre].std()))
```

LOGR: Accuracy: 0.8880 +/- 0.0174

MLP: Accuracy: 0.9146 +/- 0.0335

```
[22]: # Definimos el modelo definitivo.
algoritmos = {'LOGR': LogisticRegression(penalty='l2', solver='saga',
    ↪max_iter=1000, random_state=42),
              'MLP': MLPClassifier(hidden_layer_sizes=[8,4], activation='relu',
    ↪solver='sgd', batch_size='auto',
                                   learning_rate='adaptive', learning_rate_init=0.
    ↪01, max_iter=1000, random_state=42)}

LOGR_definitivo = LOGR.fit(X_train, y_train)
MLP_definitivo = MLP.fit(X_train, y_train)

# Atributos se obtienen durante el entrenamiento
print('Mínimo error cometido: ', MLP_definitivo.best_loss_)
print('Número de iteraciones llevadas a cabo: ', MLP_definitivo.n_iter_)
```

Mínimo error cometido: 0.1579953781086285
 Número de iteraciones llevadas a cabo: 600

```
[24]: # Guardar modelos
import os
if not os.path.exists('../models'):
    os.mkdir('../models')

import pickle
with open('../models/LOGR.pickle', 'wb') as fw:
    pickle.dump(LOGR_definitivo, fw)
with open('../models/MLP.pickle', 'wb') as fw:
    pickle.dump(MLP_definitivo, fw)
```